

ПОСОБИЕ ПО РАБОТЕ С JETVOT_MIREA

ЧАСТЬ 3. ДОПОЛНИТЕЛЬНОЕ ПО НА БАЗЕ ROS2

Разработано в РТУ МИРЭА ИИИ КПУ

Грачевым А.В.

Зениной А.А.

ОГЛАВЛЕНИЕ

Введение	4
Общая архитектура системы управления jetbot_mirea	4
Взаимодействие уровней в типовых сценариях	5
Ключевая идея: единый интерфейс через completed_scripts_jetbot	8
ROS2-пакет для моделирования jetbot_mirea - «jetbot_mirea_description» в симуляционной среде Gazebo	8
1. Короткое описание	8
Содержание пакета	9
1.1. Описание робота (URDF, SDF)	9
1.2. Готовый мир Gazebo - лаборатория Г-210	10
1.3. Launch-файлы - быстрый запуск системы	12
ROS2-пакет, ограничивающий угол обзора лидара - «lidar_filter» ..	14
1. Короткое описание	14
2. Описание	14
2.1. Нода lidar_filter	14
2.2. Launch-файл filter.launch.py	15
ROS2-пакет, содержащий URDF-описание камеры Intel RealSense D435i - «realsense_ros_gazebo»	16
1. Короткое описание	16
2. Описание	16
ROS2-пакет, реализующий последовательную отправку точек маршрута - «trajectory_maker»	17
1. Короткое описание	17
2. Описание	17
ROS2-пакет, содержащий конечные Launch-файлы, позволяющие запустить сценарии системы - «completed_scripts_jetbot»	17
1. Короткое описание	17
2. Описание	18
2.1. Launch-файл rviz_and_robot_description.launch.py	21
2.2. Launch-файл jetbot_sensors_and_motors.launch.py	21
2.3. Launch-файл navigation.launch.py	22

2.4. Launch-файл load_map.launch.py	23
3. Примечания и структура пакета	24
Обратная связь	25

Введение

Общая архитектура системы управления `jetbot_mirea`

Система управления мобильным роботом `jetbot_mirea` построена на базе ROS2 и представляет собой набор специализированных пакетов, объединённых в единую программную платформу. Для того чтобы пользователю было проще ориентироваться в многообразии компонентов, мы выделили несколько логических уровней, каждый из которых решает свою часть задач:

1. Уровень описания и симуляции

Отвечает за цифровую модель робота, его кинематику, сенсоры и визуализацию.

- Пакет: `jetbot_mirea_description` (ros2 пакет для моделирования `jetbot_mirea` - `jetbot_mirea_description` в симуляционной среде gazebo)
- Содержит URDF/SDF-описание `jetbot_mirea`, модели полигонов (лаборатория Г-210, склад), launch-файлы для запуска Gazebo и Rviz.

2. Уровень драйверов сенсоров и исполнительных устройств

Обеспечивает взаимодействие с реальным «железом»: лидаром, камерой, моторами. В симуляции их эмулируют плагины Gazebo.

- Лидар: пакет `slidar_ros2`
- Камера Intel RealSense D435i: пакет `realsense2_camera` (реальный робот) и `realsense_ros_gazebo` (симуляция)
- Моторы: пакет `serial_bridge_package` (реальный робот)
- Пакет `realsense_ros_gazebo` (ros2 пакет содержащий urdf-описание камеры intel realsense d435i - `realsense_ros_gazebo`) содержит URDF-описание камеры для использования в модели робота.

3. Уровень обработки данных

Преобразует сырые данные от сенсоров в формат, пригодный для алгоритмов навигации и SLAM.

- Пакет *lidar_filter* (ros2-пакет ограничивающий угол обзора лидара - *lidar_filter*) обрезает круговой скан лидара с 360° до 270°, убирая «мёртвую зону», занятую корпусом робота.
- Статические трансформы (TF2) между системами координат робота, включая привязку камеры.

4. Уровень навигации и SLAM

Реализует алгоритмы одновременной локализации и построения карты (SLAM), а также навигацию с учётом препятствий.

- Навигационный стек Nav2: планирование пути, управление движением, контроль скоростей.
- Пакет *slam_toolbox* для построения карты.
- Конфигурационные файлы (YAML) для настройки параметров под реального робота или симуляцию хранятся в пакете *completed_scripts_jetbot*.

5. Уровень задач и оркестрации

Обеспечивает простой запуск всей системы одной командой и позволяет ставить роботу задачи верхнего уровня.

- Пакет *completed_scripts_jetbot* (ros2-пакет содержащий конечные launch-файлы позволяющие запустить стеки системы - *completed_scripts_jetbot*) — это единая точка входа. Он содержит готовые launch-файлы, которые включают необходимые компоненты в зависимости от выбранного сценария (реальный робот / симуляция, визуализация, навигация, SLAM).
- Экспериментальный пакет *trajectory_maker* (ros2-пакет реализующий последовательную отправку точек маршрута - *trajectory_maker*) демонстрирует, как можно программно задавать маршрут (используется для отладки, не входит в основные пользовательские сценарии).

Взаимодействие уровней в типовых сценариях

Для решения конкретных задач пользователь запускает соответствующий launch-файл из пакетов *completed_scripts_jetbot* и *jetbot_mirea_description*. Все параметры (использовать ли симуляционное время, например) задаются через аргументы командной строки. Ниже приведены основные сценарии и задействованные в них компоненты.

Таблица 1. Сценарии и launch-файлы

Сценарий	Команда для запуска launch-файла	Задействованные пакеты / уровни	Краткое описание
Визуализация модели	<i>ros2 launch completed_scripts_jetbot rviz_and_robot_descriptin.launch. py</i>	<i>jetbot_mirea_description</i> (уровень 1)	Публикует трансформации робота и открывает Rviz с предустановленным видом для просмотра карты при навигации на реальном роботе.
Моделирование робота и его латчиков в среде симуляции Gazebo	<i>ros2 launch jetbot_mirea_description gazebo.launch.py</i>	<i>jetbot_mirea_description</i> (уровень 1)	Запускает симуляцию мобильного робота JetBot в Gazebo с возможностью выбора мира, типа модели робота (идеальная или реальная) и соответствующей конфигурации RViz2 для визуализации или навигации. Он также включает запуск robot_state_publisher и спавн модели робота в симуляции.
Запуск сенсоров и моторов	<i>ros2 launch completed_scripts_jetbot jetbot_sensors_and_motos.launch. py</i>	Драйверы (уровень 2), <i>lidar_filter</i> (уровень 3)	Включает лидар, камеру (опционально), моторы; данные можно наблюдать в Rviz. При остановке выполнения launch, лидар может продолжить куртиться.
Локализация на готовой карте	<i>ros2 launch completed_scripts_jetbot load_map.launch.py</i>	Уровни 1–4: описание, драйверы, фильтр, <i>nav2_map_server, nav2_amcl</i>	Загружает карту из файла map.yaml, запускает AMCL для определения положения робота. Аргумент <i>config_choice</i> определяет, используются параметры для реального робота (1) или симуляции (2).

Запуск навигационного стека	<i>ros2 launch completed_scripts_jetbot navigation.launch.py</i>	Уровни 1–4: описание, драйверы, фильтр, Nav2 (<i>planner, controller, recovery</i>)	Запускает планировщик маршрута, контроллер движения и прочие компоненты Nav2. Может использоваться как самостоятельно (если карта и локализация уже запущены), так и в комбинации с <i>load_map.launch.py</i> . Параметры навигации выбираются аргументом <i>config_choice</i> .
Построение карты (SLAM)	<i>ros2 launch completed_scripts_jetbot mapping.launch.py</i>	Уровни 1–4: описание, драйверы, фильтр, <i>slam_toolbox</i>	Запускает SLAM Toolbox для создания карты помещения. Аналогично выбирается конфигурация <i>real/sim</i> .
Автономное движение по маршруту	(отдельный запуск <i>trajectory_maker</i>)	Уровень 5 + активный стек навигации	Экспериментальный режим: после запуска навигации (например, <i>navigation.launch.py</i>) можно отправить роботу последовательность точек.

Ключевая идея: единый интерфейс через `completed_scripts_jetbot`

Все пользовательские **launch-файлы** находятся в пакетах **`completed_scripts_jetbot`** и **`jetbot_mirea_description`**. Это сделано для того, чтобы избавить пользователя от необходимости запоминать, в каком пакете лежит тот или иной файл, и как правильно связывать компоненты.

Внутри этих launch-файлов с помощью `IncludeLaunchDescription` подключаются необходимые подсистемы, а параметры (например, пути к конфигурационным YAML-файлам) автоматически подставляются в зависимости от значения аргумента `config_choice` (1 - реальный робот, 2 - симуляция). Это гарантирует, что в симуляции будет использовано время Gazebo (`use_sim_time=true`), а для реального робота - реальное время, и что настройки навигации будут соответствовать выбранному режиму.

Для удобства переконфигурирования в launch-файлах были созданы аргументы (аналог параметров). Посмотреть аргументы launch-файла можно добавив ключ «-s» в команду запуска. Например:

```
ros2 launch jetbot_mirea_description gazebo.launch.py -s
```

Таким образом, пользователь всегда работает с одним пакетом-оркестратором и пакетом моделирования и не вникает в детали внутреннего устройства остальных компонентов, если это не требуется для разработки или отладки.

Примечание: Пакет `trajectory_maker` является вспомогательным и предназначен для тестирования навигационной системы; в стандартные launch-файлы он не включён.

ROS2-пакет для моделирования `jetbot_mirea` - «`jetbot_mirea_description`» в симуляционной среде Gazebo

1. Короткое описание

Это ROS2-пакет, для моделирования робота `jetbot_mirea`. `jetbot_mirea` - это модель описанная с помощью URDF, соответствующая массо-габаритным характеристикам робота и

эмулирующая работу всех ключевых датчиков. Эмулируются такие датчики как обе камеры, лидар, IMU, энкодеры на моторах и MarvelMind (реализовано GPS, вместо MarvelMind, из-за сходства технологий). Эмулируется так же шум от датчиков. Одновременно с симуляцией реализован запуск Rviz, для просмотра показаний с перечисленных датчиков. Для упрощенного запуска предоставлены launch-файлы, который так же позволяют запустить разные полигоны.

Также в симуляционной среде воссоздан полигон - лаборатория Г-210, точнее, полигон для jetbot с кубическими подвижными препятствиями.

Содержание пакета

В пакете содержатся следующие модули:

- 3D-модель полигона аудитории-лаборатории Г-210 в blender;
- launch-файлы для запуска публикации описания робота в топик (URDF), запуска симуляции;
- 3D-модели частей jetbot (в настоящий момент не используются);
- конфигурации запуска Rviz (для симмуляции и для реального робота);
- URDF-описание робота, SDF-дополнение для моделирования в среде моделирования Gazebo;
- готовый мир с полигоном из аудитории-лаборатории Г-210 для среды моделирования Gazebo.
- готовый мир - простой лабиринт для среды моделирования Gazebo.

Рассмотрим некоторые модули подробнее:

1.1. Описание робота (URDF, SDF)

Описание робота делится на два подмодуля:

- 1) URDF-описание - описывает трансформации между системами координат робота, а также его статичные (звенья - links) и динамичные (кинематические пары - joints) элементы.
- 2) SDF-описание - описывает плагины и свойства для нормальной симуляции в среде моделирования Gazebo. Оно задаёт физические свойства свойства (например, трение) звеньям и кинематическим парам. Также осуществляет возможность

моделирования датчиков (камера глубины, камера, лидар и т.д.) и исполнительные элементы (например, дифференциальный привод).

Хасро — это инструмент командной строки в ROS 2 для упрощения URDF-файлов, позволяющий объявлять переменные, использовать математические выражения и создавать параметризованные макросы для переиспользуемых частей робота. Это избавляет от дублирования кода и громоздких цифр, делая описание робота более модульным, гибким и читаемым. Файлы .хасро затем автоматически преобразуются в стандартный URDF для симуляции.

Реализованы 2 версии jetbot_mirea - идеальная и реалистичная. Отличаются они тем, что реалистичная модель имеет касторы чуть меньше под высоте чем идеальная. Из-за этого торможении и старте реалистичная модель заваливается назад или вперёд. Такое поведение соответствует поведению реального робота на полигоне в Г-210, но искажает показания лидара и усложняет картографирование местности.

1.2. Готовый мир Gazebo - лаборатория Г-210

Полгон повторяет рельные размеры и масштаб лаборатории Г-210. Фото модели полигона представлено на рисунке 1. Позицию кубов можно изменять. Так же они двигаются если воздействовать на них роботом. Можно добавить еще таких же препятсвий-кубов с помощью панели слева путём перетаскивания.

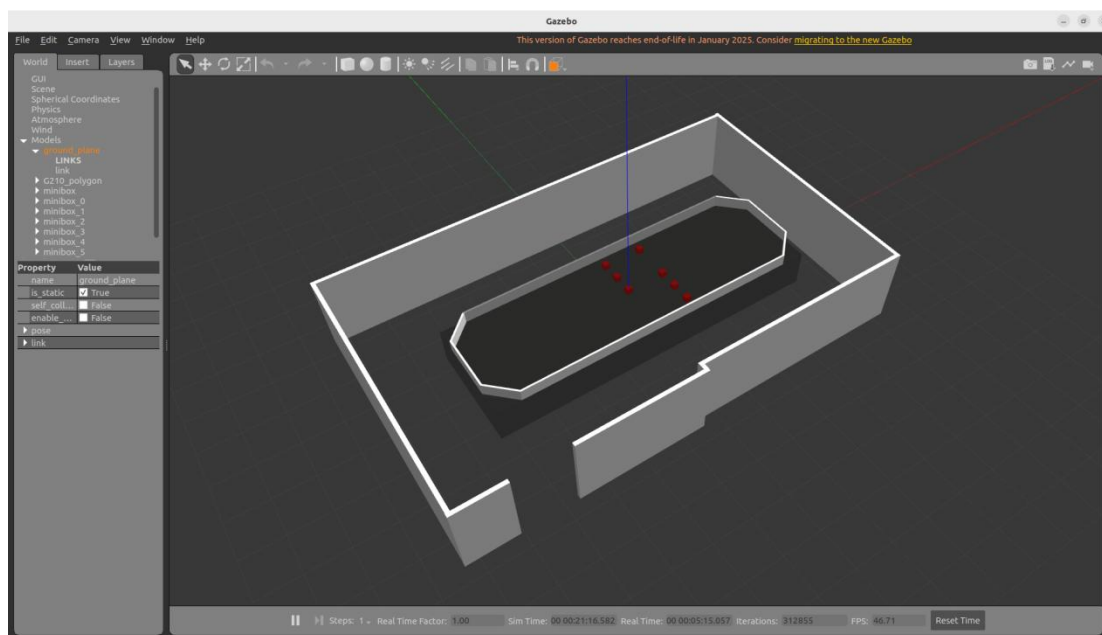


Рисунок 1 - Полигон Г-210 в Gazebo

Для добавления новых препятствий - кубиков - необходимо во вкладке «insert» нажать на блок «minibox», после установить кубик в любое место на полу (все кубики изначально появляются на нулевой высоте). Далее выбрав инструмент для переноса «Translation Mode» (на верхней панели с инструментами) перенести кубик в нужное место на полигоне. Изображения 2-3 - иллюстрируют добавление новых препятствий.

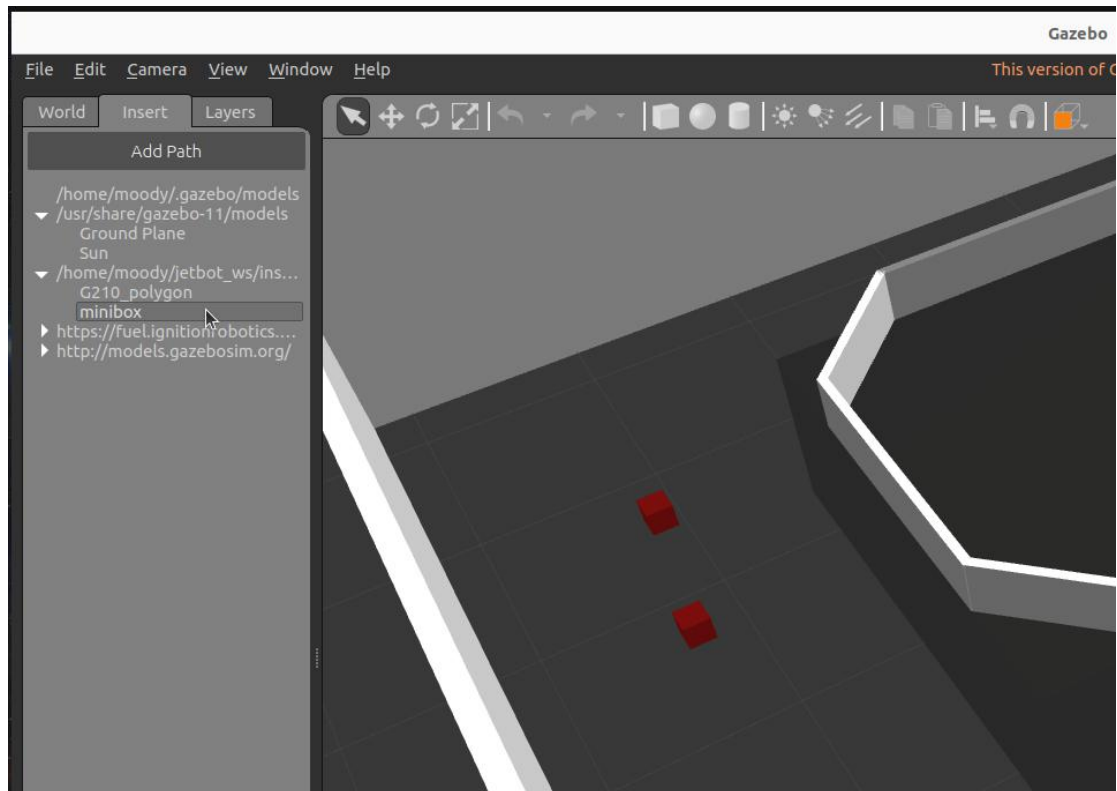


Рисунок 1 - Во вкладке «insert» нажатие на блок «minibox».

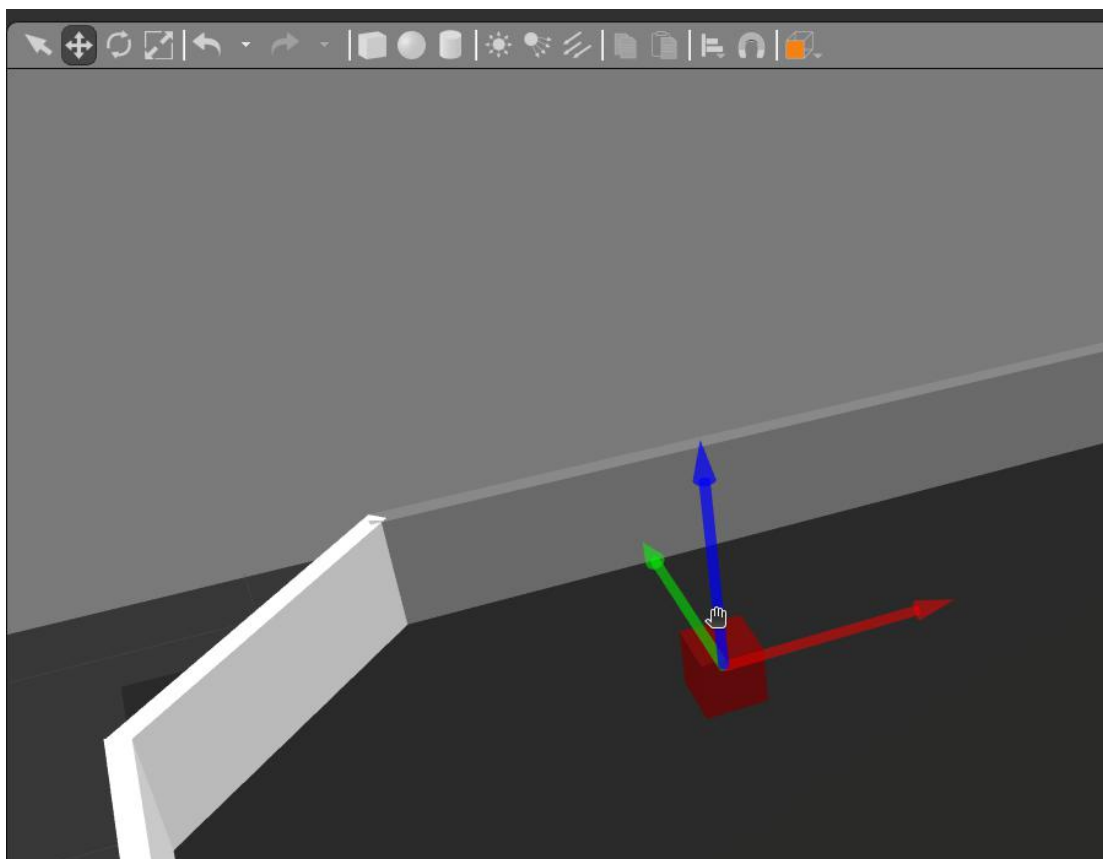


Рисунок 3 - Перенос кубика с помощью «Translation Mode».

1.3. Launch-файлы - быстрый запуск системы

Для быстрого запуска всей системы были созданы launch-файлы:

1. *real_robot.launch.py* - Этот launch-файл запускает *robot_state_publisher* и *joint_state_publisher* для публикации состояния реального робота JetBot на основе URDF-модели, а также опционально запускает RViz2 с предварительно настроенной конфигурацией для визуализации. Он не включает симуляцию Gazebo и предназначен для работы с реальным оборудованием. Запуск осуществляется следующей командой:

```
ros2 launch jetbot_mirea_description real_robot.launch.py
```

2. *gazebo.launch.py* - запускает симуляцию мобильного робота JetBot в Gazebo с возможностью выбора мира (например, аудитория Г210 с препятствиями), типа модели робота (идеальная или реальная) и соответствующей конфигурации RViz2 для визуализации или навигации. Он также включает

запуск `robot_state_publisher` и спавн модели робота в симуляции. Запуск осуществляется следующей командой:

```
ros2 launch jetbot_mirea_description gazebo.launch.py
```

Для удобства переконфигурирования в `launch`-файлах были созданы аргументы (аналог параметров). Посмотреть аргументы `launch`-файла можно добавив ключ «-s» в команду запуска. Например:

```
ros2 launch jetbot_mirea_description gazebo.launch.py -s
```

В таблицах 1 и 2 представлены аргументы `launch`-файлов `real_robot.launch.py` и `gazebo.launch.py` соответственно.

Таблица 2. Аргументы `launch`-файла `real_robot.launch.py`.

Аргумент	Тип	Значение по умолчанию	Пояснения
<code>launch_rviz_description</code>	Bool	True	Запускает / не запускает RViz2. Возможны варианты: «True», «False».

Таблица 2. Аргументы `launch`-файла `gazebo.launch.py`.

Аргумент	Тип	Значение по умолчанию	Пояснения
<code>launch_rviz</code>	bool	True	Запускает / не запускает RViz2. Возможны варианты: «True», «False».
<code>model_use</code>	string	ideal	Позволяет выбрать между реальной и идеальной моделью jetbot. Возможны варианты: «ideal», «real».
<code>world_id</code>	int	2	ID полигона: 1 - Г210, 2 - Г210 с препятствиями, 3 - складской. Возможны варианты: «1», «2», «3».

ROS2-пакет, ограничивающий угол обзора лидара - «lidar_filter»

1. Короткое описание

Это ROS2-пакет, обеспечивающий обрезку лидарного скана с 360° до 270°, убирая «мёртую зону» - область, где робот видит сам себя.

2. Описание

Во время работы, лидар считывает информацию вокруг себя (все 360°), есть сектор, в котором лидар определяет части jetbot как препятствия, это ломает алгоритмы SLAM. Нода обрезает скан с 360° до 270°, убирая сектор с корпусом робота из зоны сканирования лидара.

В пакете содержится 1 нода (lidar_filter.py) и 1 launch-файл (filter.launch.py), который позволяет запустить ноду с некоторой конфигурацией.

2.1. Нода lidar_filter

Узел ROS 2 для фильтрации данных лазерного скана (LaserScan) по угловому сектору. Конфигурируется параметрами: задает входной/выходной топики, границы сектора в градусах и режим инверсии. Использует QoS с политикой BEST_EFFORT для совместимости с лидарами.

В таблице 2 представлены используемые интерфейсы в системе ROS2. Значения указанные в таких скобках: <>, являются параметрами.

Таблица 3. Используемые интерфейсы ROS2 в ноде.

Тип взаимодействия	Название	Тип сообщения	Комментарий
Подписка	<input_topic>	sensor_msgs/msg/LaserScan	Исходный скан
Публикация	<output_topic>	sensor_msgs/msg/LaserScan	Обрезанный скан

В таблице 3 представлены параметры ноды.

Таблица 4. Параметры ноды.

Имя параметра	Тип	Значение по умолчанию	Комментарий
start_angle_deg	float	-135.0	Начало сектора в градусах
end_angle_deg	float	135.0	Конец сектора в градусах
input_topic	string	/scan	Имя входного ROS2-топика
output_topic	string	/scan_filtered	Имя выходного ROS2-топика
invert_sector	bool	False	Инвертировать сектор (сохранить все, кроме указанного)
output_frame_id	string		идентификатор_фрейма для темы вывода

Запуск осуществляется командой

```
ros2 run lidar_filter lidar_filter
```

2.2. Launch-файл *filter.launch.py*

В таблице 4 представлено содержание launch-файла, точнее запускаемые ноды и их параметры.

Таблица 5. Содержание launch-файла.

Нода	Параметры
lidar_filter	<output_frame_id> <start_angle_deg> <end_angle_deg> <input_topic> <output_topic> <invert_sector> <use_sim_time>

Аргументы launch-файла во многом совпадают с параметрами ноды по названию и смыслу. Есть 1 дополнительный параметр: «use_sim_time» типа bool. По умолчанию его значение в этом файле задаётся как «True», может быть «True», «False». Если значение «True», то нода использует время симуляции, если «False» - реальное.

Запуск осуществляется командой:

```
ros2 launch lidar_filter filter.launch.py
```

ROS2-пакет, содержащий URDF-описание камеры Intel RealSense D435i - «realsense_ros_gazebo»

1. Короткое описание

Это ROS2-пакет, содержащий URDF-описание камеры Intel RealSense D435i, а также SDF-описание для моделирования в Gazebo.

2. Описание

Внутри камеры глубины Intel RealSense D435i находятся несколько датчиков, системы координат которых статически неизменны относительно друг друга, но они находятся внутри корпуса, что является сложностью в нахождении преобразования между координатами. Данную проблему уже решил автор этого пакета, выложивший его на Github. Также автор добавил все сенсоры, находящиеся внутри камеры глубины в SDF-описание, что позволяет моделировать полный функционал в среде моделирования Gazebo. Мы использовали этот пакет для того чтобы в модель jetbot_mirea добавить камеру глубины Intel RealSense D435i.

Ссылка на Github-репозиторий пакета представлена ниже
https://github.com/nilseuropa/realSense_ros_gazebo/tree/humble

ROS2-пакет, реализующий последовательную отправку точек маршрута - «trajectory_maker»

1. Короткое описание

Этот ROS2-пакет предоставляет набор нод для задания навигационных целей мобильному роботу (в частности, jetbot_mirea). Ноды поддерживают два режима работы: ручной ввод координат цели с клавиатуры и автоматическую публикацию заранее определённой последовательности точек маршрута. В пакете реализованы два различных подхода к взаимодействию с навигационной системой: через публикацию в топик /goal_pose (для простых систем) и через ROS 2 Action Client с использованием стандартного интерфейса NavigateToPose из Nav2.

2. Описание

Пакет состоит из двух исполняемых нод: start_move_topic и start_move_action. Обе ноды написаны на Python и используют библиотеку rclpy. В составе пакета отсутствуют launch-файлы, запуск нод производится непосредственно через ros2 run. Пакет не имеет внешних зависимостей, кроме стандартных для навигации (nav2_msgs, geometry_msgs). Последовательность точек маршрута жёстко задана в исходном коде (хардкод) и может быть изменена только путём редактирования файлов нод.

ROS2-пакет, содержащий конечные Launch-файлы, позволяющие запустить сценарии системы - «completed_scripts_jetbot»

1. Короткое описание

Пакет completed_scripts_jetbot представляет собой набор готовых launch-файлов, объединяющих запуск всех необходимых компонентов системы управления мобильным роботом jetbot_mirea.

Он служит единой точкой входа для различных сценариев работы: визуализация модели робота, управление сенсорами и моторами, навигация с использованием готовой карты, построение карты местности (SLAM), а также локализация.

Пакет не содержит исполняемых узлов, но включает конфигурационные файлы (YAML) для настройки параметров навигации, SLAM, AMCL и камеры RealSense, а также RViz-конфигурации и заранее подготовленные карты помещений. Благодаря этому пакету пользователь может быстро развернуть всю систему как на реальном роботе, так и в симуляции Gazebo, выбирая нужный режим через аргументы запуска.

2. Описание

Пакет построен как набор launch-файлов, каждый из которых отвечает за определённую функциональность. Все они используют механизм OpaqueFunction для реализации условной логики и подключают необходимые подсистемы через IncludeLaunchDescription. Конфигурационные параметры вынесены в отдельные YAML-файлы, расположенные в подкаталоге config, а файлы карт – в подкаталоге maps. Визуализация в RViz2 обеспечивается готовым файлом конфигурации full.rviz в каталоге rviz

Таблица 6. Сценарии и launch-файлы

Сценарий	Команда для запуска launch-файла	Задействованные пакеты / уровни	Краткое описание
Визуализация модели	<i>ros2 launch completed_scripts_jetbot rviz_and_robot_descriptin.launch.py</i>	<i>jetbot_mirea_description</i> (уровень 1)	Публикует трансформации робота и открывает Rviz с предустановленным видом для просмотра карты при навигации на реальном роботе.
Моделирование робота и его датчиков в среде симуляции Gazebo	<i>ros2 launch jetbot_mirea_description gazebo.launch.py</i>	<i>jetbot_mirea_description</i> (уровень 1)	Запускает симуляцию мобильного робота JetBot в Gazebo с возможностью выбора мира, типа модели робота (идеальная или реальная) и соответствующей конфигурации RViz2 для визуализации или навигации. Он также включает запуск robot_state_publisher и спавн модели робота в симуляции.
Запуск сенсоров и моторов	<i>ros2 launch completed_scripts_jetbot jetbot_sensors_and_motos.launch.py</i>	Драйверы (уровень 2), <i>lidar_filter</i> (уровень 3)	Включает лидар, камеру (опционально), моторы; данные можно наблюдать в Rviz. При остановке выполнения launch, лидар может продолжить куртиться.
Локализация на готовой карте	<i>ros2 launch completed_scripts_jetbot load_map.launch.py</i>	Уровни 1–4: описание, драйверы, фильтр, <i>nav2_map_server</i> , <i>nav2_amcl</i>	Загружает карту из файла map.yaml, запускает AMCL для определения положения робота. Аргумент <i>config_choice</i> определяет, используются параметры для реального робота (1) или

			симуляции (2).
Запуск навигационного стека	<i>ros2 launch completed_scripts_jetbot navigation.launch.py</i>	Уровни 1–4: описание, драйверы, фильтр, Nav2 (<i>planner, controller, recovery</i>)	Запускает планировщик маршрута, контроллер движения и прочие компоненты Nav2. Может использоваться как самостоятельно (если карта и локализация уже запущены), так и в комбинации с <i>load_map.launch.py</i> . Параметры навигации выбираются аргументом <i>config_choice</i> .
Построение карты (SLAM)	<i>ros2 launch completed_scripts_jetbot mapping.launch.py</i>	Уровни 1–4: описание, драйверы, фильтр, <i>slam_toolbox</i>	Запускает SLAM Toolbox для создания карты помещения. Аналогично выбирается конфигурация <i>real/sim</i> .
Автономное движение по маршруту	(отдельный запуск <i>trajectory_maker</i>)	Уровень 5 + активный стек навигации	Экспериментальный режим: после запуска навигации (например, <i>navigation.launch.py</i>) можно отправить роботу последовательность точек.

Ниже подробно описаны launch-файлы пакета.

2.1. Launch-файл *rviz_and_robot_description.launch.py*

Данный launch-файл предназначен для запуска визуализации модели робота и его систем координат. Он включает:

- Описание робота из пакета `jetbot_mirea_description` (запускает `jetbot_mirea_rviz.launch.py`);
- Статический трансформ между фреймами `realsense_link` и `camera_link` (необходим для корректной привязки камеры);
- RViz2 с предварительно настроенным отображением (`full.rviz`).

Файл не имеет объявленных аргументов и при одиночном запуске предназначен для быстрой проверки модели робота и трансформаций. В общем используется в стэке навигации для публикации трансформ (TF2), просмотра карты и модели робота.

Запуск осуществляется командой:

```
ros2 launch completed_scripts_jetbot  
rviz_and_robot_description.launch.py
```

2.2. Launch-файл *jetbot_sensors_and_motors.launch.py*

Этот файл запускает все аппаратные сенсоры и приводы робота. Он позволяет гибко включать/отключать отдельные компоненты через аргументы. Предназначены для запуска непосредственно на вычислителе робота, к которому подключены перечисленные основные запускаемые подсистемы:

- Лидар (пакет `slidar_ros2`) и его фильтр (пакет `lidar_filter`);
- Фронтальная камера Intel RealSense D435i (пакет `realsense2_camera`) с пользовательской конфигурацией;
- Моторы через последовательный мост (пакет `serial_bridge_package`).
- Также добавляется статический трансформ между фреймами камеры (аналогично предыдущему файлу) при включённой камере.

**При остановке выполнения этого launch-файла лидар может продолжить крутиться.*

В таблице 9 представлены аргументы launch-файла `jetbot_sensors_and_motors.launch.py`.

Таблица 7. Аргументы launch-файла jetbot_sensors_and_motors.launch.py.

Аргумент	Тип	Значение по умолчанию	Пояснения
enable_lidar	bool	True	Включить/отключить лидар и его фильтр.
enable_front_camera	bool	False	Включить/отключить камеру Intel RealSense D435i.
enable_motors	bool	True	Включить/отключить управление моторами через serial bridge.
lidar_serial_port	string	/dev/rplidar	Путь к последовательному порту лидара.
angle_compensate	bool	true	Включить компенсацию угла сканирования (поддерживается драйвером лидара).
scan_mode	string	Standard	Режим сканирования лидара: Standard (2K точек), Express (4K), Boost (8K).

Запуск осуществляется командой:

```
ros2 launch completed_scripts_jetbot
jetbot_sensors_and_motors.launch.py
```

2.3. Launch-файл navigation.launch.py

Запускает навигационный стек Nav2 с выбором конфигурации для реального робота или симуляции. Использует пакет nav2_bringup (launch-файл navigation_launch.py) и передаёт ему путь к соответствующему файлу параметров.

В таблице 11 представлены аргументы launch-файла navigation.launch.py.

Таблица 8. Аргументы launch-файла navigation.launch.py.

Аргумент	Тип	Значение по умолчанию	Пояснения
config_choice	string	1	Выбор конфигурации: 1 – для реального робота (navigation_param_real.yaml), 2 – для симуляции (navigation_param_sim.yaml).

Для симуляции запуск осуществляется командой:

```
ros2 launch completed_scripts_jetbot mapping.launch.py
config_choice:=2
```

Для реального робота запуск осуществляется командой:

```
ros2 launch completed_scripts_jetbot mapping.launch.py
config_choice:=1
```

или

```
ros2 launch completed_scripts_jetbot mapping.launch.py
```

2.4. Launch-файл load_map.launch.py

Обеспечивает локализацию робота на заранее построенной карте. Запускает сервер карт (nav2_map_server) с загрузкой карты из файла map.yaml, узел AMCL (nav2_amcl) с соответствующими параметрами (real/sim), а также менеджер жизненного цикла (nav2_lifecycle_manager) для автоматического запуска узлов.

Особенности: карта загружается с задержкой 2 секунды (через TimerAction) для синхронизации запуска. Аргумент use_sim_time объявлен, но в текущей реализации жёстко привязан к выбору конфигурации: для реального робота (config_choice=1) use_sim_time=false, для симуляции (config_choice=2) use_sim_time=true.

В таблице 12 представлены аргументы launch-файла load_map.launch.py.

Таблица 9. Аргументы launch-файла load_map.launch.py.

Аргумент	Тип	Значение по умолчанию	Пояснения
config_choice	string	1	Выбор конфигурации: 1 – для реального робота (navigation_param_real.yaml), 2 – для симуляции (navigation_param_sim.yaml).
use_sim_time	bool	false	(Объявлен, но фактически не используется – значение определяется через config_choice.)

Для симуляции запуск осуществляется командой:

```
ros2 launch completed_scripts_jetbot load_map.launch.py
config_choice:=2
```

Для реального робота запуск осуществляется командой:

```
ros2 launch completed_scripts_jetbot load_map.launch.py
config_choice:=1
```

или

```
ros2 launch completed_scripts_jetbot load_map.launch.py
```

3. Примечания и структура пакета

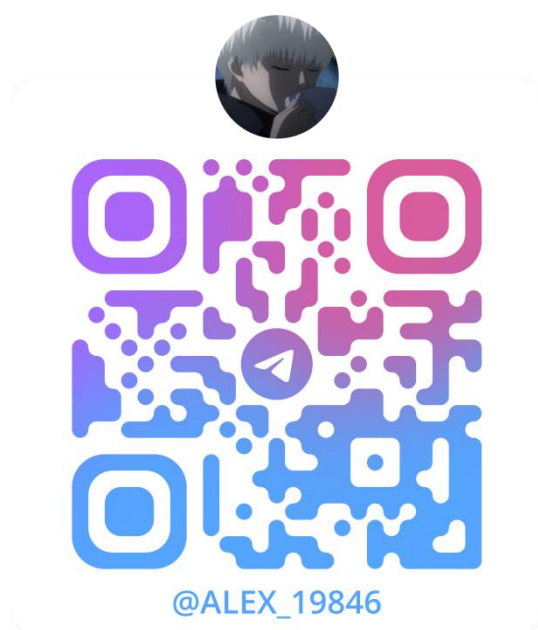
- Конфигурационные файлы (каталог config):
 - ❑ navigation_param_real.yaml, navigation_param_sim.yaml – параметры Nav2.
 - ❑ mapper_params_real.yaml, mapper_params_sim.yaml – параметры SLAM Toolbox.
 - ❑ amcl_config_real.yaml, amcl_config_sim.yaml – параметры AMCL.
 - ❑ realsense_config.yaml – пользовательские настройки камеры RealSense.
- RViz-конфигурация (каталог rviz): full.rviz – предустановленный вид для визуализации всех компонентов.

- Карты (каталог maps): map.yaml (и соответствующий файл изображения) – готовая карта помещения (лаборатория Г-210 с полигоном или с препятствиями).

Обратная связь

Если Вы нашли ошибку, неточность, у Вас есть предложения по улучшению или вопросы, напишите в Telegram Александру (https://t.me/Alex_19846) или Алисе (https://t.me/Kika_01). QR-коды со ссылками на их аккаунты представлены ниже.

Александр



Алиса

